Qsimov library for neural networks

The Qsimov library is a Python library that, starting from a given feed-forward neural network that uses the rectified linear unit (ReLU) activation function on its hidden layers, enables the definition of a new AI system and associated inference and training methods. The main benefit of the new AI system is extremely fast re-training since incremental re-training does not suffer from catastrophic forgetting.

1. Installation

1.1 Using a virtual environment

We recommend using a Python virtual environment to install and manage dependencies for this library. A virtual environment is an isolated environment where you can install Python packages without affecting your system's global Python installation.

To create a new virtual environment, you can use the `venv` module. For this project, we used python 3.8.10, so we recommend using this version of Python to create the virtual environment. First install python 3.8, and then create the virtual environment using:

```
/path/to/python3.8 -m venv myenv
```

This will create a new virtual environment named 'myenv' in your current directory.

Next, activate the virtual environment:

- On Linux: `source myenv/bin/activate`

1.2 Installing the dependencies

Once you've activated the virtual environment, you can install the required dependencies using the 'requirements.txt' file included with the library:

```
pip install -r requirements.txt
```

This will install all the necessary packages required for using this library. You can now start using the library within the virtual environment. Remember to activate the virtual environment every time you want to use the library. Take into account that the requirements.txt file includes both keras and pytorch libraries, but you only need to install one of them, depending on the backend you want to use, unless you want to use both backends, or run the tests. You may update the contents of the requirements.txt file to omit pytorch or keras if you only want to use one of them.

We have included a 'requirements-dev.txt' file that includes the packages required for tasks such as documentation generation and code styling. You can install these packages using:

```
pip install -r requirements_dev.txt
```

To use this library with a GPU you must follow the steps described by the corresponding Tensorflow/Pytorch documentation (depending on the backend you want to use) that would normally be required to use a GPU with Tensorflow/Pytorch.

1.3 Installing this library

You must also install the source code of this library. You can do this by running the following command:

```
pip install -e /path/to/qsimov/
```

This will install the library in editable mode, so any changes you make to the source code will be reflected in the library. You can import the library in your Python scripts using:

```
import qsimov
```

1.4 Compiling the cython modules and testing

This library uses Cython to improve performance. Cython is a programming language that is a superset of Python, which allows you to write C extensions for Python. Once you've activated the virtual environment, you can compile the Cython modules required by running the following command:

```
python setup.py build_ext --inplace
```

This will build the necessary Cython modules.

1.5 Testing

To run the tests of this library to ensure all is working as expected, run the tests using:

```
pytest tests/
```

2. Documentation

The new AI system handles structural and quantitative information separately. Structural information is stored in and handled by the path selector, which contains a neural network and some additional logic. Structural information is initialized by pre-training the neural network but it is not updated when re-training. The quantitative information is stored in the estimator, which consists in the collection of paths in the path selector's neural network, each of them having an assigned weight. The path weights are updated when the system is re-trained. In order to have control on the number of paths in the estimator, the neural network is split into two parts (left and right). Only the right part is used in the path selector and to extract the collection of paths.

2.1 Generating the API documentation

To generate the documentation of the API you can install pydoctor (preferably in a virtual environment) running the following command:

```
pip install pydoctor
```

Once installed, you can generate the documentation by running the following command:

```
pydoctor
```

Which will output HTML documentation in the `docs` folder, which can be opened in a browser (open `docs/index.html`).

2.2 Examples of usage

This library implements two main algorithms, Qsimov with linear system solving, and Qsimov with gradient descent. The first one only works for the mean squared error loss function, while the second one works for any loss function. We have provided examples for each of these in the `tutorials/` folder.

The API involves functions defined in the `qsimov` module, which can be documented as explained in 2.1.

An example of usage script is provided for each algorithm and framework. The scripts that demonstrate the API are located in the `tutorials/api/` folder, note that to run them, you need to install requirements_dev.txt.

In `tutorials/api/(tf_keras|pytorch)/qsimov_linear_system.py` there is an example of how to use the Qsimov algorithm with linear system solving on the California Housing dataset for a regression problem.

In `tutorials/api/(tf_keras|pytorch)/qsimov_gradient.py` there is an example of how to use the Qsimov algorithm with gradient descent on the MNIST dataset for a multiple label classification problem, using the categorical cross entropy loss function.

2.3 Description of the algorithm

2.3.1 Path selection

Both of the methods described below require a previously trained neural network ϕ on some data $\{X, Y\}$, where X is some input data and Y is the corresponding output data. We will denote the output of the neural network ϕ on an input sample x as $\hat{y} = \phi(x)$. Each component i of a vector x is denoted as x^i, with $i = 1, ..., n$, where n is the number of input components of ϕ. Each component o of the output $\hat{y}$ is denoted as $\hat{y}^o$, with $o = 1, ..., d$ and d the number of outputs of ϕ.

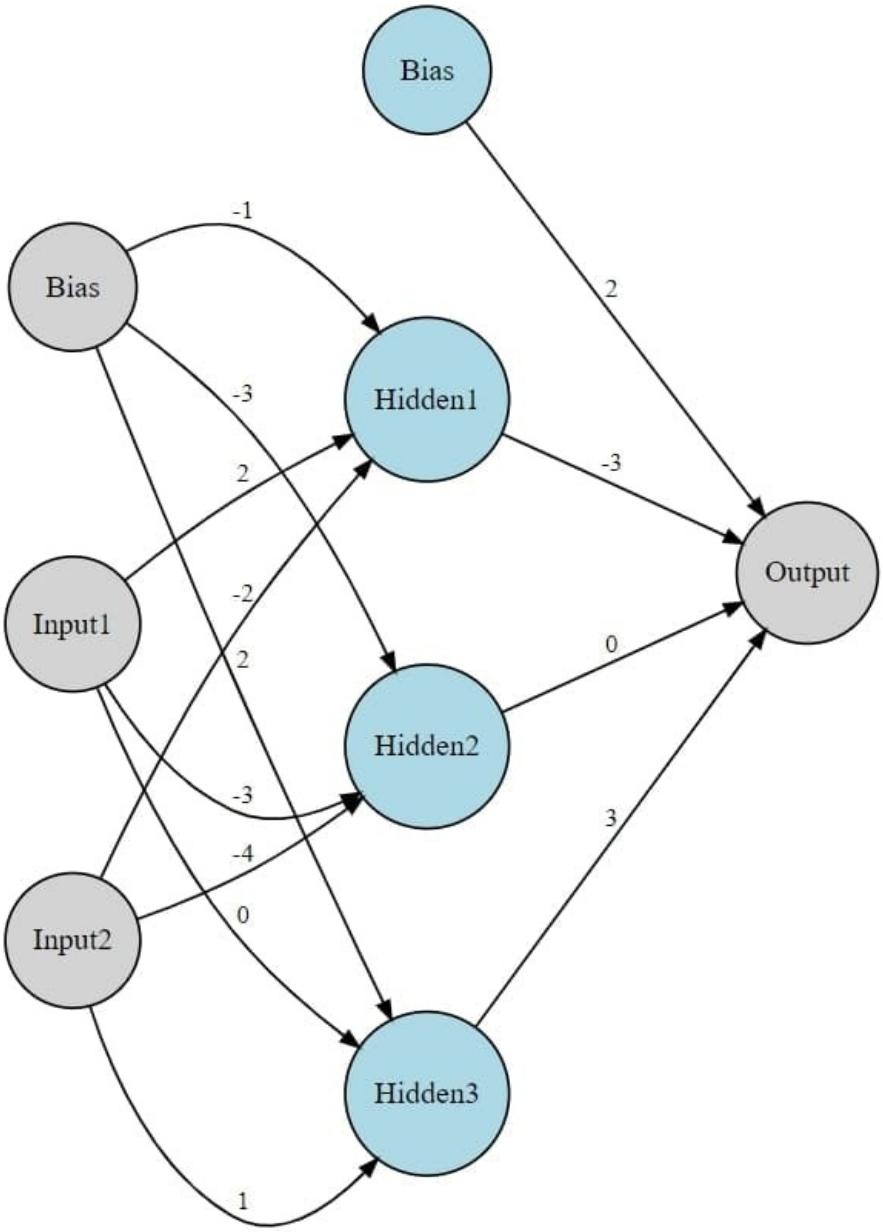
Given the feed-forward neural network ϕ, and a layer index l, we can split it into two neural networks ϕ_L (which includes up to layer $l-1$) and ϕ_R, which includes the remaining layers from l onwards (with l as the input layer), with $l \in \{0, ..., \lambda - 2\}$, where λ is the total number of layers of the neural network ϕ (including the input layer). The inputs of ϕ_L will be the same as the inputs of ϕ, and the outputs of ϕ_R will be the same as the outputs of ϕ. The outputs of ϕ_L will be the inputs of ϕ_R. So the following holds: $\phi(x) = \phi_R(\phi_L(x))$. We will denote the output of ϕ_L on an input sample x as $\hat{y_L} = x_R = \phi_L(x)$, and the output of ϕ_R on an input sample x_R as $\hat{y_R} = \phi_R(x_R)$. Note that for $l = 0$, ϕ_L is the identity function, but by the upper bound of l, ϕ_R always has at least two layers (including the input layer).

The Qsimov method will focus on the neural network ϕ_R, which is where path selection will be performed. The neural network ϕ_L will be kept fixed during training and is only used to generate the inputs of ϕ_R. For this neural network ϕ_R, we denote λ_R as the number of layers of ϕ_R (including the input layer), and it holds that $\lambda_R = \lambda - l$. The main reason for splitting the original neural network is that the cost of the algorithm grows exponentially with the number of layers of the right neural network ϕ_R, so we want to keep this network as small as possible.

The neural network ϕ_R must always verify that all its layers use either the ReLU or the linear activation function

(identity function), except for the last one, which may use any activation function if the Qsimov gradient algorithm is applied. If the Qsimov linear system solving method is used, the last layer of ϕ_R must use the linear activation function.

To better understand the following explanations, examples will be provided in paragraphs like this one based on an example ϕ_R with three layers. The first one is an input layer with 2 neurons, the second one is a fully-connected layer with 3 neurons and the last one has 1 neuron. Additionally, the second and the third layer have a ReLU activation function.



First, **paths** must be defined. The right neural network ϕ_R can be seen as a directed acyclic graph, where each node is a neuron, and each edge is a connection (weight) between two neurons. We define that two neurons are not connected if the weight between them is zero, or the connection pattern of the layer does not link them (e.g. convolutional layers, inbound connections to bias neurons...). A **path** p is a vector of indices $p = (p^0, p^1, ..., p^{\lambda_R - 1})$. Each index p^i represents a neuron of the i-th layer of the right neural network ϕ_R. Each index goes from 0 to N_i, where N_i is the number of neurons in the i-th layer of ϕ_R and $p^i = 0$ corresponds to the bias neuron of the i-th layer of ϕ_R. A path can only exist if each successive pair of indices p^i and p^{i+1} are connected in ϕ_R. As there are paths that begin in bias neurons of hidden layers of ϕ_R, we define them to be padded with zeros on the left.

In our example network ϕ_R, a possible path is $p = (1, 2, 1)$, which corresponds to the path that goes from the first neuron of the first layer to the second neuron of the second layer, and then to the first neuron of the third layer. An example of a path that goes from the bias in the hidden layer to the neuron in the output layer is $p = (0, 0, 1)$, where the first zero, which corresponds to the input layer, is a padding added according to the previously stablished definition of paths; even if the bias neurons between layers are not connected.

For the sake of simplicity, we will assume that ϕ_R only contains dense layers. Each hidden layer and output layer of ϕ_R

has a weight matrix and bias vector, denoted respectively as W^i and b^i, with i being the index of the layer. The weight matrix W^i is a matrix of size $N_{i-1} \times N_i$, where N_{i-1} is the number of neurons in the previous layer, and N_i is the number of neurons in the current layer. The bias vector b^i is a vector of size N_i. The output of ϕ_R can be defined as:

$$\hat{y}_R = \phi_R(x_R) = f_{\lambda_R-1}(...(f_2(f_1(x_R W^1 + b^1)W^2 + b^2)...W^{\lambda_R-1} + b^{\lambda_R-1}))$$

where f_i is the activation function of the i-th layer of ϕ_R, which we have defined to be either ReLU or the identity function for $i < \lambda_R - 1$.

We can define the weights for our sample network ϕ_R as:

$$W^1 = \begin{pmatrix} 2 & -3 & 0 \\ -2 & -4 & 1 \end{pmatrix}$$

$$b^1 = (-1, -3, 2)$$

$$W^2 = \begin{pmatrix} -3 \\ 0 \\ 3 \end{pmatrix}$$

$$b^2 = (2)$$

Given that we are using dense layers, each weight and bias matrix implicitly represents the adjacency matrix of the layer. For example, the second row of W^1 represents the connections of the second neuron of the first layer (input layer) of ϕ_R to the neurons in the second layer (dense) of ϕ_R. It can be seen that $w^1_{13} = 0$, and $w^2_{21} = 0$ meaning that paths with $p^0 = 1$ and $p^1 = 3$ or $p^1 = 2$ and $p^2 = 1$ are not possible, such as $p = (1, 3, 1)$ or $p = (1, 2, 1)$. Moreover, paths such as $p = (1, 0, 1)$ are not possible as the bias neuron of the second layer is not connected to the first neuron of the first layer.

More specifically, from all the possible paths for a network with this structure,
$\{(0,0,1),(0,1,1),(0,2,1),(0,3,1),(1,1,1),(1,2,1),(1,3,1),(2,1,1),(2,2,1),(2,3,1)\}$, only the paths
$\{(0,0,1),(0,1,1),(0,3,1),(1,1,1),(2,1,1),(2,3,1)\}$ are possible due to zeros in the weight matrices.

As f_1 and f_2 are the ReLU function, we can define

$$\hat{y}_R = \phi_R(x_R) = ReLU(ReLU(x_R W^1 + b^1)W^2 + b^2)$$

If we define an input $x_R = a^0 = (1, 2)$, we can calculate the second layer activations:

$$a^1 = ReLU(x_R W^1 + b^1) = (0, 0, 4)$$

And the activation of the third layer that is equivalent to the output $\hat{y}_R$:

$$\hat{y}_R = a^2 = ReLU(a^1 W^2 + b^2) = (14)$$

The Qsimov method relies on an alternative representation of the previous equation. We can also define the output of the *o*-th component of $\hat{y}_R$ as the sum of the contributions of all the paths leading to it:

$$\hat{y}^o_R = f_{\lambda_R-1}(\sum_{p \in P_o} x_R^{p^0} \alpha(p) \delta(x_R, p))$$

which will be denoted as a **path based forward pass (PBFP)** where P_o is the set of all paths of ϕ_R that end in the *o*-th component of $\hat{y}_R$. $x_R^{p^0}$ represents the p^0-th component of x_R, meaning that for each element of the sum, we take the component of x_R that corresponds to the beggining of the path p. We define the weight of path p, $\alpha(p)$, as the product of the corresponding neuron weights along path p, meaning that $\alpha(p) = \prod_{i=1}^{|p|-1} \omega(p^{i-1}, p^i)$. The ω is an auxiliar function to choose either weight or bias depending on the connection $p^{i-1} \to p^i$:

$$\omega(p^{i-1}, p^i) = \begin{cases} w^i_{p^{i-1}p^i} & \text{if } p^{i-1}, p^i >= 1 \\ b^{p^i} & \text{if } p^{i-1} = 0, p^i >= 1 \\ 1 & \text{otherwise} \end{cases}$$

Finally, to account for the possibility of using the ReLU activation function, we define $\delta(x_R, p)$ as 1 if the path p is active for x_R, and 0 otherwise. A path p is active if and only if, after propagating x_R through ϕ_R using a standard forward pass, on each layer i of ϕ_R with $i = 0, ..., \lambda_R - 2$ (we do not take the output layer into account), the neuron p^i is active. A neuron p^i is active if and only if the component p^i of the output of the i-th layer of ϕ_R, a^i, is greater than 0, meaning

that the ReLU activation function is active for that neuron. This process of masking out inactive paths is called **path selection**.

In our example network, there is only one output, so $P_1 = \{(0,0,1),(0,1,1),(0,3,1),(1,1,1),(2,1,1),(2,3,1)\}$. *Let's see an active path and an inactive path in the previous example.*

With $x_R = (1,2)$, *the activation* $a^1 = (0,0,4)$ *indicates that paths with* $p^1 = 1$ *or* $p^1 = 2$ *are inactive because the first and second components* a_1^1, a_2^1 *are* 0. *Therefore the paths* $(0,1,1)$, $(1,1,1)$ *and* $(2,1,1)$ *are inactive. In the delta function, this translates to* $\delta(x_R,(0,1,1)) = \delta(x_R,(1,1,1)) = \delta(x_R,(2,1,1)) = 0$.

The rest of the paths are active, so $\delta(x_R,(0,0,1)) = \delta(x_R,(0,3,1)) = \delta(x_R,(2,3,1)) = 1$.

Note that the activations of the last layer a^2 *are not taken into account in the delta function. We neither need to take into account* $a^0 = x_R$, *as* x_R *already multiplies the delta in the PBFP.*

Naturally, to compute $\hat{y}_R^o$ using the PBFP, we need to compute $\delta(x_R,p)$ for each path p of ϕ_R, which at the same time requires computing the forward pass $\hat{y}_R = \phi_R(x_R)$, using the classical forward pass. Therefore, to compute $\hat{y}_R^o$ using the PBFP, we also need to compute $\hat{y}_R$ in some other way, and so it appears we have not gained anything. However, the goal of the Qsimov method is not to replace the classical forward pass, but to define a replacement network ϕ_Q for ϕ_R in a way that makes use of the PBFP and enables incremental re-training.

First, it should be noted that the PBFP expression for $\hat{y}_R^o$ is identical to the expression for the activation of a single neuron whose weights are defined by $\alpha(p)$, for $p \in P_o$, and each individual input is masked by $\delta(x_R,p)$ (since $\delta(x_R,p)$ can only take binary values). Therefore, each component $\hat{y}_R^o$ of $\hat{y}_R$ can be modeled using a single neuron. This model can be extended to consider all components of $\hat{y}_R$ by modeling each component with a different neuron. The resulting model is a flat (single-layer) neural network.

By using this model, the PBFP can be expressed to compute all components of $\hat{y}_R$ in a single matrix product, using some new definitions:

$$\hat{y}_R = f_{\lambda_R - 1}(x_Q W_Q) = \phi_Q(x_Q)$$

which is equivalent to the forward pass of a flat (no hidden layers) neural network ϕ_Q where the output layer is dense with a weight matrix W_Q and an activation function f_{λ_R-1}, and where x_Q is the input of the flat network.

In the previous definition of the PBFP, which was made for each separate output o, we use each path $p \in P_o$, meaning the paths that end in the o-th component of $\hat{y}_R$. However, now we take each path $p \in P$, with P being the set of all paths of ϕ_R that start at the input layer and end at the **penultimate** layer of ϕ_R. The cardinality of said set $|P|$ is the upper bound of $|P_o|$, where $|P_o| < |P|$ when the output neuron o is not connected to all the previous neurons. The reason to only use paths up to the penultimate layer is that the last layer is not taken into account in the delta function, and so, all the delta values are independent of the last component of the path. We also include in P a padding path $(0,0,...,0)$ to account for paths such as $(0,0,...,0,o)$ in P_o, which exist when the output neuron o is connected to a bias neuron in the last layer.

Each component of x_Q is defined as:

$$x_Q^p = x_R^{p^0}\delta(x_R,p)$$

where we define $x_R^0 = 1$, to account for $p^0 = 0$, and $x_Q^{(0,...,0)} = 1$ to account for padding paths in P.

If we want ϕ_Q to produce the same outputs as ϕ_R, each element w_{po} of W_Q is defined as:

$$w_{po} = \alpha((p,o))$$

where (p,o) is the concatenation of the index of an output neuron o to the path p, i.e., $(p,o) = (p^0,...,p^{\lambda_R-2},o)$. (p,o) is essentially extending the path p to reach the neuron o of the output layer of ϕ_R. It is used as a convention that when using a path p to index the weight matrix, a biyection is made between said path and an integer representing that path in the corresponding set of paths P.

In our example network, we had $P_1 = \{(0,0,1),(0,1,1),(0,3,1),(1,1,1),(2,1,1),(2,3,1)\}$. *However, when defining* P, *paths are defined up to the penultimate layer, meaning that* $P = \{(0,0),(0,1),(0,2),(0,3),(1,1),(1,2),(2,1),(2,2),(2,3)\}$. *The cardinality of* P *is* $|P| = 9$, *which is the upper bound of* $|P_1| = 6$.

If we wanted ϕ_Q to produce the same outputs as ϕ_R, following the order of paths $P = \{(0,0),(0,1),(0,2),(0,3),(1,1),(1,2),(2,1),(2,2),(2,3)\}$, the W_Q matrix would be defined as:

$$W_Q = \begin{pmatrix} \alpha((0,0,1)) \\ \alpha((0,1,1)) \\ \alpha((0,2,1)) \\ \alpha((0,3,1)) \\ \alpha((1,1,1)) \\ \alpha((1,2,1)) \\ \alpha((2,1,1)) \\ \alpha((2,2,1)) \\ \alpha((2,3,1)) \end{pmatrix} = \begin{pmatrix} b_1^2 \\ b_1^1 \cdot w_{11}^2 \\ b_2^1 \cdot w_{21}^2 \\ b_3^1 \cdot w_{31}^2 \\ w_{11}^1 \cdot w_{11}^2 \\ w_{12}^1 \cdot w_{21}^2 \\ w_{21}^1 \cdot w_{11}^2 \\ w_{22}^1 \cdot w_{21}^2 \\ w_{23}^1 \cdot w_{31}^2 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ 0 \\ 6 \\ -6 \\ 0 \\ 6 \\ 0 \\ 3 \end{pmatrix}$$

The values of x_Q are computed as:

$$x_Q = (1, x_R^0 \cdot \delta(x_R,(0,1)), x_R^0 \cdot \delta(x_R,(0,2)), x_R^0 \cdot \delta(x_R,(0,3)), x_R^1 \cdot \delta(x_R,(1,1)), ..., x_R^2 \cdot \delta(x_R,(2,3))) = (1, 1 \cdot 0, 1 \cdot 0, 1 \cdot 1, 1 \cdot 0, 1 \cdot 0, 2 \cdot 0, 2 \cdot 0, 2 \cdot 1) = (1,0,0,1,0,0,0,0,2)$$

Note that the first element is 1 to account for the padding path $(0,0)$.

The output of ϕ_Q is computed as:

$$\phi_Q(x) = ReLU(x_Q W_Q) = ReLU(14) = 14$$

We will define a **path selector** ρ_ϕ as a function that takes a sample x as an input and returns a vector x_Q, i.e., $\rho_\phi(x) = x_Q$. To do so, the sample is propagated through the left neural network ϕ_L to obtain x_R and then x_Q is computed using ϕ_R.

We had previously defined W_Q so that $\phi_R(x_R) \equiv \phi_Q(\rho_\phi(x))$, but now, any W_Q will be allowed. The training algorithms aim at finding the optimal weights W_Q of ϕ_Q in a way that some loss function L is minimized on some new data $\{X', Y'\}$. The underlying idea for the Qsimov algorithms is that the weights of ϕ_L and ϕ_R are not changed during training, only those of the new network ϕ_Q. In other words, the path selector does not change during training, and therefore, the active paths for each sample remain unaltered. Despite this, the new network ϕ_Q learns by training its weights W_Q.

The Qsimov linear system algorithm provides a direct solution (i.e. it is not iterative) and can be used when the loss function L is the mean squared error, and the final layer does not have any activation function. The Qsimov gradient descent algorithm can be used when the loss L is any differentiable function, and also enables the final layer to incorporate any activation function.

2.3.2 Qsimov training method based on solving a linear equation system

Inputs:

- Neural network ϕ, pre-trained with some previous data using standard gradient descent.
- New data $\{X, Y\}$.
- Initial layer l.
- System equations solver (back substitution or least squares algorithm from numpy).
- Shrinkage factor: Ratio between the number of equations (which is proportional to the number of samples) and the number of paths per output (i.e. number of trainable parameters) in the linear system.

Outputs:

- W_Q, the weight matrix of the one-layer neural network ϕ_Q.

Algorithm:

1. Build path selector ρ_ϕ, from initial layer l onwards.

2. Initialize linear system set for all outputs o:

$$S = \{(A_o, b_o), |A_o| = |b_o| = 0, \forall o\}$$

3. For each new training data $\{X, Y\}$:

   3.1. Consider $x_Q = \rho_\phi(x)$ and $X_Q = \{\rho_\phi(x), \forall x \in X\}$.

3.2. From X_Q, get the coefficient matrix for the linear equations for each output o: $X_{Q,o}$.

3.3. Append the new equations for each output o to the previous set of equations for the linear system:

$$A_o = A_o \oplus X_{Q,o}$$

$$b_o = b_o \oplus Y_o$$

3.4. To bound memory usage: For any A_o whose number of rows is larger than the number of columns (number of paths per output) times the shrinkage factor, a QR factorization is applied on the horizontal concatenation of A_o and b_o, $(A_o|b_o)$ to avoid too large systems. **Note**: Only the upper triangular matrix R of the QR is calculated. When performing QR on the concatenation of A and b, the last row of R can be removed, and we can take the triangular matrix A' consisting of the first $n-1$ columns of R, where n is the number of columns of R, and the vector b' consisting of the last column of R. Later, at step 5 of this algorithm, we can solve $A'x = b'$ using back substitution. This is equivalent to solving the linear system $Ax = b$. The last row of R is removed beacuse it only contains the accumulation of error for the solution. This error is present when the system is overdetermined, meaning that there are more equations than unknowns.

4. If not performed when processing the last $\{X, Y\}$ data (due to shrinkage factor criteria), apply QR factorization to $(A_o|b_o)$ for each o, getting the matrix R_o.

5. Solve each R_o, using back-substitution as detailed in step 3.4. Each solution is a column of the weight matrix W_Q.

6. Go to step 3 for new data. If there are no more data, return the weight matrix W_Q.

2.3.3 Qsimov training method based on gradient descent

Inputs:

- Neural network ϕ, pre-trained with some previous data using standard gradient descent.
- New data $\{X, Y\}$.
- Initial layer l.

Outputs:

- W_Q, the weight matrix of the one-layer neural network ϕ_Q.

Algorithm:

1. Build path selector ρ_ϕ, from initial layer l onwards.

2. Create one-layer dense neural network ϕ_Q, with as many inputs as the maximum of paths to one output, and as many outputs as the original network ϕ.

    - Note: The connection pattern of this layer is masked as necessary, depending on the connection pattern (weights set to zero, or sparse connection pattern, e.g. convolutional layer) of the last layer in ϕ_R. We filter out paths that are not present in the different outputs if those paths include a connection in the last layer of ϕ_R that does not exist due to its connection pattern.

3. After mapping the training dataset X with the path selector ρ_ϕ to the inputs of ϕ_Q, we fit ϕ_Q using a gradient descent based algorithm (SGD, Adam, ...). This way, the weights of W_Q are estimated.